Introduction

Programming: Everyday Decision-Making Algorithms

LECTURER

Dr. Nils Roemer, External Lecturer

AFFILIATION

Planningio GmbH

Today in one sentence

In this course you learn how computer scientists approach everyday decisions, and how to turn these ideas into Python code.

WHY IT MATTERS

More and more business decisions rely on data and algorithms. You don’t need to become a programmer, but understanding how algorithms think, and writing a few lines of code yourself, makes you a better decision maker. No prior knowledge required.

Roadmap

01 About this course: topics, format and how you pass

02 How to learn programming

03 Setting up Python

Six topics, one per session

- **I:** Optimal Stopping
- **II:** Explore & Exploit
- **III:** Caching
- **IV:** Scheduling
- **V:** Randomness
- **VI:** Computational Kindness

Take part, it pays off

- **Participate actively**: it makes the course easier and more fun
- Inspired by the book **Algorithms to Live By**[1]
- All material is online: courses.nilsroemer.com

1. Christian, B., & Griffiths, T. (2016). Algorithms to live by: the computer science of human decisions. First International edition. New York, Henry Holt and Company.

Speaker notes

Today’s agenda in detail: about the course and expectations, how to learn programming (mindset and resources), setting up Python with uv and VS Code, using notebooks with uv, Q&A and next steps.

Each session: lecture, then tutorial

- **Lecture:** we discuss an algorithm for everyday decisions
- **Tutorial:** step-by-step assignments, solved together in groups
- **Difficulty:** depends on your programming experience



No worries, we will help you out if you have any questions!

Speaker notes

Lecture: presentation and discussion of algorithms related to everyday decision-making. Tutorial: step-by-step assignments to be solved and discussed together in groups. Difficulty: strongly depends on your background and programming experience.

How you pass

- **Pass/fail course** without exams
- **75% attendance** required
- **Two assignments** on algorithmic problems from the course
- **Short final presentation** of your prototype in the last lecture
- You can work in groups of **up to three students**

How to hand in assignments

- Each group submits **one solution** per assignment
- Hand in your **working notebooks**
- Due by the **end of the quarter**, earlier is better
- At least **50%** of the solutions have to be correct

Speaker notes

Earlier submissions are highly encouraged: you get feedback while the topic is still fresh.

You learn Python in the tutorials

We will mostly **not cover Python during the lectures!**

Question: Does anybody know why?

- The best way to learn is by **doing**
- Lectures focus on the decision-making algorithms
- You learn Python by **doing the tutorials**

Speaker notes

In our experience, the best way to learn programming is by doing. Don’t worry, we will help you out if you have any questions.

Programming is like a new language

- At first it might be **a little bit overwhelming**
- First you get used to it and learn words
- Later you apply it and see results
- Important: *Practice, practice, practice!*

What you take home

- The **basics of programming**
- **Algorithmic thinking**: how computer scientists approach a problem
- Methods you can **apply** to practical problems

Why Python?

- **Simple syntax**: easy to read and to learn
- **Versatile**: web, data analysis, artificial intelligence
- **Big community**: help and documentation everywhere

Speaker notes

Origins: Python was conceived in the late 1980s as a teaching and scripting language. Its syntax is mostly straightforward and very easy to learn. It is used in web development, data analysis, artificial intelligence and more, and it has a large community of users worldwide with extensive documentation.

AI is allowed, but learn yourself

- You are **allowed to use AI**, we use it as well
- Assistants such as Claude, ChatGPT or Gemini are **great for learning Python**

> ⚠ But don't let them *replace* your learning.

Speaker notes

Current AI assistants help you a lot to get started with programming. But you should not simply use them to replace your learning: read and understand what they suggest.

Roadmap

01 About this course: topics, format and how you pass

02 **How to learn programming**

03 Setting up Python

Three habits that work

1. **Be present**: attend the lecture and solve the tutorials
2. **Put in some work**: repeat code and try to understand it
3. **Do coding**: run code, play around, modify, and solve

Speaker notes

Great resources to start are books (see the list at the end of the lecture) and small challenges, for example on Codewars.

Don’t give up!

- Programming is **problem solving**, errors are normal
- Expect to **stretch** your comfort zone



Roadmap

01 About this course: topics, format and how you pass

02 How to learn programming

03 **Setting up Python**

VS Code is where you write code

- Like **Word for text**, but for code: an app to **write, run and find mistakes** in code
- Apps like this are called **IDEs** (Integrated Development Environments)
- We use VS Code for the **whole course**

Speaker notes

Other IDEs exist, for example PyCharm or Zed. We use VS Code because it is free, works on all systems and has good notebook support. Every step of the setup is in the Installation Guide on the website, with a check at the end of each step.

Install VS Code

- Download from code.visualstudio.com, for **Windows and Mac**
- Open it once: you should see a **welcome page**

> Every step is in the Installation Guide, with a check at the end.

Add the Python and Jupyter extensions

- Out of the box, VS Code doesn’t know Python. Extensions **teach it**
- Click the **Extensions** icon on the left (four small squares)
- Install **Python** and **Jupyter**, both by Microsoft

Create your course folder

- One folder for everything, called `programming-course`
- In your **home folder**, not in OneDrive
- In VS Code: **File** → **Open Folder...** and select it

Speaker notes

Python works best when each project has its own folder. OneDrive syncing can break the setup on Windows.

The terminal: typing instead of clicking

- A window where you **type commands** and the computer answers with text
- Open it in VS Code: **Terminal → New Terminal**
- Copy a command from the guide, paste it, press **Enter**, wait until the line is back

Speaker notes

On a Mac the line ends with a percent sign, on Windows it starts with PS for PowerShell. We only need a handful of commands, all of them are on the guide page to copy.

Install `uv`

- `uv` installs Python and everything Python needs, you install it **once**

macOS:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Then **close VS Code completely**, open it again, and check: `uv --version`

Speaker notes

Closing and reopening is important: only then does the terminal know the new tool. If uv is “not found”, that is almost always the reason.

Install Python

- Everyone uses the **same version**, so the code works the same for all

```
1 uv python install 3.13
```



If the installation does not work, **let us know!**

Turn your folder into a Python project

In the terminal, inside `programming-course`, one after the other:

```
uv init
```

```
uv python pin 3.13
```

Check: `uv run python main.py` prints `Hello from programming-course!`

Speaker notes

uv creates a few files: main.py, pyproject.toml and .python-version. Nobody needs to touch them.

Add notebook support

```
uv add --dev ipykernel
```

```
uv add jupyter
```

- The first lets notebooks use **the Python of your project**, the second adds the **notebook tools**

Speaker notes

A new folder .venv appears: the project’s own Python environment. Done? Perfect, now we can start.

Your first notebook

- **Cmd + Shift + P** (Mac) or **Ctrl + Shift + P** (Windows), then `Create: New Jupyter Notebook`
- **Select Kernel** in the top right: **.venv** with **Python 3.13**
- Type `print("Hello")` and press **Shift + Enter**

> Something not working yet? Ask us, or see “Something doesn’t work?” in the guide.

Download a tutorial and open it

- Open a tutorial on the website and click **Jupyter** under **Other Formats**
- Move the file into `programming-course`
- Open it in VS Code and choose the **.venv** kernel



That was the hardest part today!

What we covered

ABOUT THIS COURSE: TOPICS, FORMAT AND HOW YOU PASS

Six topics, each with a lecture and a Python session. You pass with 75% attendance, two assignments and a final presentation.

HOW TO LEARN PROGRAMMING

Programming is learned by doing: small steps, regular practice, and don’t give up.

SETTING UP PYTHON

Your laptop is ready: VS Code, Python with uv and notebooks.

Why it matters

More and more business decisions rely on data and algorithms. You don’t need to become a programmer, but understanding how algorithms think, and writing a few lines of code yourself, makes you a better decision maker.

Learn by doing.

Any questions so far?

After the break: Optimal Stopping

- The “Secretary Problem” and the 37% rule
- When to stop searching and make a decision
- Testing the rule ourselves with a small simulation



That’s it for our introduction!
Let’s have a short break and then continue with our first topic.

Literature

Interesting literature to start

- Christian, B., & Griffiths, T. (2016). Algorithms to live by: the computer science of human decisions. First international edition. New York, Henry Holt and Company.[1]

1. The main inspiration for this lecture. We have read it and discussed it in depth, always wanting to translate it into a course.

Books on Programming

- Downey, A. B. (2024). Think Python: How to think like a computer scientist (Third edition). O'Reilly. Free online
- Elter, S. (2021). Schrödinger programmiert Python: Das etwas andere Fachbuch (1. Auflage). Rheinwerk Verlag.



Think Python is free online and a great start. Schrödinger programmiert Python is a playful alternative in German.

More Literature

For more interesting literature, take a look at the literature list of this course.